STAT 1010 Lecture Notes

Yi Wang

2023-08-11

Table of contents

Preface **6**

1 Introduction **7**

2 Setting-up Python Computing Environment **8**
2.1 Use Google Colab 8
2.1.1 How to run a project file from your Google Drive? 8
2.2 On your own computer 9

3 Setting-up R Studio Computing Environment **10**
3.1 Setting up your own computing environment on a personal computer 10
3.2 Use R-Studio Cloud (No setting-up needed) 10

4 Use Git and GitHub **11**
4.1 Download Git 11
4.2 Establish a connection between a local repo and a remote GitHub repo 11
4.2.1 Clone an existing repo on GitHub 11
4.2.2 Initializing a Git Directory Locally First 12
4.3 Some other common commands 14
4.4 Use Git help 16
4.5 When the upstream repo changes 16
4.6 Create branch 17
4.7 Merge branch to main branch 17
4.8 Handle large files (>= 150Mb) on GitHub 17
4.9 Contribute by forking a GitHub repo and commit to the forked repo and create a pull request (refer to [the best workflow below]Section 6.3) 18
4.10 Project 19
4.11 More on git 20
4.12 Git pull: What does `--ff` mean? 20
4.12.1 `git pull` or `git pull --ff` (merge fallback) 21
4.12.2 Option 2: `git pull --rebase` (**replay** your work on top of remote) . . 22
4.13 How to set opitons gloabally 22

5 Concrete example: what does "merge C with D to produce M" look like? **23**
5.0.1 Commits and changes 23
5.0.2 You run: `git pull` (merge strategy) or `git merge origin/main` 23

6 3) What is `git push --force-with-lease` (and why it's safer than `--force`)? 25
6.0.1 Typical rebase + push flow . . . 25
6.0.2 Example workflow with `git stash` . . . 25
6.1 Rebuild the index respecting `.gitignore` . . . 28
6.2 Unstage and untrack . . . 28
6.2.1 To **unstage** (but keep tracking): . . . 28
6.2.2 To **unstage**: . . . 28
6.2.3 Prevent tracking in the future . . . 29
6.3 Best workflow with GitHub from Colab (or a local device) . . . 29
6.4 Team Github workflow . . . 30
6.5 Initial setup . . . 30
6.6 Keep your fork in sync . . . 31
6.7 Git FAQ . . . 31
6.7.1 Working directory (working tree) vs "actual files on disk"? Save vs commit? What are "index" and "working tree"? . . . 32
6.7.2 1) After `git add`, how to undo (un-add) a file or directory? . . . 33
6.7.3 2) After `git commit`, how to un-commit? . . . 33
6.7.4 3) When `git push`, what conflicts can occur? How to fix them? . . . 34
6.7.5 4) When `git pull`, what conflicts can occur, and how to fix them? . . . 35
6.7.6 5) Why create a new branch instead of working on `main`? . . . 36
6.7.7 6) How `git stash` works and why we need it . . . 36
6.7.8 8) Difference between `git reset` and `git revert` . . . 37
6.7.9 9) How to remove files that are already pushed? Explain `git rm --cached` 37
6.7.10 10) Difference between `git pull --rebase` and `git pull -ff` . . . 38
6.7.11 11) Explain `git rebase` . . . 38
6.8 4) Difference between `git rebase` and `git merge` . . . 39
6.8.1 Merge . . . 39
6.8.2 Rebase . . . 39
6.8.3 On which branch do `merge` and `rebase` happen? . . . 40
6.9 Quick reference (handy snippets) . . . 40
6.10 1) Index vs. working files (aka working tree) . . . 41
6.11 3) "I saved a file on one branch, then checked out a new branch and edited it again. What version do I have on disk?" . . . 42
6.11.1 Cases . . . 42
6.11.2 Tips . . . 42
6.12 4) Suggested team workflow (you maintain `main`, teammates contribute) . . . 43
6.12.1 Repository / policy (one-time setup) . . . 43
6.12.2 Personal Git config (everyone) . . . 43
6.12.3 Contributor workflow (feature branch) . . . 43
6.12.4 Maintainer (you) merging PRs . . . 44
6.12.5 Hotfixes . . . 44
6.12.6 Common "gotchas" and fixes . . . 44
6.12.7 Quick reference of commands mentioned . . . 45

6.13 How to fix conflict when switching branches . . . 45
6.14 When conflict occurs using git pull and how to resolve it . . . 46
6.15 Conflicts occur when git push and how to resolve . . . 47
6.16 Git merge a branch . . . 49
6.17 resolve a file conflict after `git pull`: . . . 49
6.18 1) See what's conflicted . . . 49
6.19 2) Open the conflicted file and resolve . . . 50
6.20 3) Mark it resolved and continue . . . 50
6.20.1 Fast shortcuts (use with care) . . . 51
6.20.2 Sanity checks . . . 51
6.20.3 B) Abort the in-progress operation, then pull clean . . . 51
6.21 Steps to fix conflict: . . . 52
6.22 Option A: Rebase (keeps linear history) . . . 53
6.23 Option B: Merge (creates a merge commit) . . . 53
6.24 Option C: Fast-forward only (fail if divergence exists) . . . 54
6.25 "I just want to discard my local commits and match remote" . . . 54
6.26 Workfow when devloping codes on a local computer: . . . 54
6.26.1 most common case: fast-forward . . . 54
6.26.2 If `main` and `dev` diverged . . . 55
6.26.3 Notes . . . 55
6.27 `git add` . . . 55
6.27.1 rebase . . . 56

7 5) `git stash -u -m "message"` and `git stash pop` 57
7.1 Mental model (the "three trees") . . . 58
7.2 How it's used . . . 58
7.3 Diff cheat-sheet . . . 58
7.4 Merges & conflicts . . . 58
7.5 Partial commits . . . 58
7.6 Where it lives / advanced bits . . . 59
7.6.1 Explain `git restore --staged <path>` or `git reset <path>` . . . 59
7.7 Common scenarios & exact commands . . . 59
7.7.1 1) I ran `git add -A` by accident and staged too much . . . 59
7.7.2 2) I staged a deletion, but I want the file back . . . 60
7.7.3 3) I want the index for this file to match some other commit (not HEAD) 60
7.7.4 4) I want to discard *both* staged and working-tree changes (revert file fully) 60
7.7.5 5) Interactive unstage by hunks . . . 60
7.8 When to prefer which . . . 60
7.9 Quick reference table . . . 61

8 Hwo to go back to a previous commit? 62

9 1) Just look around at an old commit (temporary) 63

10 2) Make a new branch starting from that commit **64**

11 3) Restore a single file as it was in that commit **65**

12 4) Move the current branch pointer back (rewrite history) **66**

13 5) "Undo" by adding inverse commits (safe for shared repos) **67**

14 My Jupyter Notebook **68**
- 14.0.1 Perform addtion . . . 68
- 14.0.2 Horizontal Rule . . . 68
- 14.0.3 Bulet list . . . 69
- 14.0.4 Numbered list . . . 69
- 14.0.5 Tables . . . 69
- 14.0.6 Hyperlinks . . . 69
- 14.0.7 Images . . . 69
- 14.0.8 Code/Syntax highlighting . . . 69
- 14.0.9 Blocked quotes . . . 70
- 14.0.10 Strikethrough . . . 70

15 Homework Assignments **71**

References **73**

Preface

This is a book for STAT 1010: Introduction to Data Science at Auburn University at Montgomery. The book is written using Quarto.

To learn more about Quarto books visit https://quarto.org/docs/books.

1 Introduction

This is a book for STAT 1010: Introduction to Data Science offered at Auburn University at Montgomery.

This an ongoing project and updates are perpetually added.

2 Setting-up Python Computing Environment

2.1 Use Google Colab

All you need is a Google account. Sign in your Google account in a browser, and navigate to Google Colab. Google Colab supports both `Python` and `R`. `Python` is the default engine. Change the engine to `R` in `Connect->change runtime type`. Then you are all set. Your file will be saved to your Google Drive or you can choose to send it to your `GitHub` account (recommended).

2.1.1 How to run a project file from your Google Drive?

Many times, when you run a python file in Colab, it needs to access other files, such as data files in a subdirectory. In this case, it would be convenient to have the same file structure in the Google Colab user home directory. To do this, you can use Google Drive to store your project folder, and then mount the Google Drive in Colab.

Let's assume the project folder name, `pydata-book/`.Here are the steps:

1. `git clone` the project folder (example: `git clone https://github.com/wesm/pydata-book.git`) to your local folder. This step is only needed when you want to clone some remote repo from GitHub.
2. **Upload** the folder (ex: `pydata-book`) to Google Drive.
3. **Open the file using Colab**. In Google Drive, double click on the `ipynb` file, example, `ch06.ipynb` (or click on the three dots on the right end, and choose `open with`, then `Google Colaborotary`), the file will be opened by Google Colab.
4. **Mount the Google Drive**. In Google Colab, with the specific file (example, `ch06.ipynb`) being opened, move your cursor to the first code cell, and then click on the folder icon (this should be the fourth icon) on the upper left border in the Colab browser. This will open the file explorer pane. Typically you would see a folder named `sample_data` shown. On the top of the pane, click on the Google Drive icon to mount the Google Drive. Google Colab will insert the following code below the cursor in your opened `ipynb` file:

```
from google.colab import drive
drive.mount('/content/drive')
```

Run this code cell by pressing `SHIFT+ENTER`, and follow the prompts to complete the authentication. Wait for ~10 seconds, your Google Drive will be mounted in Colab, and it will be displayed as a folder named `drive` in the file explorer pane. You might need to click on the `Refresh` folder icon to see the folder `drive`.

5. Open a new code cell below the above code cell, and type the code

```
%cd /content/drive/MyDrive/pydata-book/
```

This is to change the directory to the project directory on the Google Drive. Run this code cell, and you are ready to run the file `ch06.ipynb` from the folder `pydata-book` on your personal Google Drive, just like it's on your local computer.

2.2 On your own computer

1. **Anaconda**: Download anaconda and install using default installation options
2. **VSC**: Download VSC and install
3. start VSC and install VSC extensions in VSC: Python, Jupyter, intellicode
4. (optional) **Quarto** for authoring: Download Quarto and install
5. Start an anaconda terminal. Navigate to the file directory.
6. Setup a conda **virtual environment**: stat1010 and install python and ipykernel engines

   `conda create -n stat1010 python ipykernel`
7. Activate the venv: `conda activate stat1010`
8. start VSC by typing `code .` in the anaconda terminal.
9. open/create a `.ipynb` or `.py` file.
10. Select the kernel `stat1010`
11. Run a code cell by pressing `Shift+Enter` or click the triangular play button.
12. Continue to run other cells.
13. After finishing using VSC, close the VSC, and deactivate the virtual environment in a conda terminal: `conda deactivate`

3 Setting-up R Studio Computing Environment

3.1 Setting up your own computing environment on a personal computer

This is the recommended way and the advantage is that it's easy to handle files.

- Go to the website <https://posit.co/download/rstudio-desktop/>.
- Follow the two steps:
    1. download and install R: Choose the appropriate operating system, and then choose "base" to "install R for the first time". You can simply accept all default options.
    2. download Rstudio Desktop and Install it.

After installation, start R-Studio, and you are ready to use it.

3.2 Use R-Studio Cloud (No setting-up needed)

Alternatively, one can save the hassle of setting up on a personal computer and use the R-Studio Cloud for **free**. Here are the steps:

- Go to the website https://login.rstudio.cloud.
- Either create a new account using an email address such as your AUM email or simply "Log in using Google" or click on other log-in alternative.

After log-in to your account, you are ready to use R Studio.

4 Use Git and GitHub

I assume you already have an account on https://github.com. If not, you need to create an account there.

4.1 Download Git

1. Go to the website https://git-scm.com/downloads, select an appropriate operating system, select "Click here to download"
2. Run the downloaded setup file with a name such as `Git-2.42.0.2-64-bit.exe`, and accept all default options.

4.2 Establish a connection between a local repo and a remote GitHub repo

4.2.1 Clone an existing repo on GitHub

This is an easier way to establish a connection between a local repo and a remote repo if the remote repo is created ahead. We will make a connection between a remote repo in your GitHub account and a local directory. If the remote repo is not under your account, then skip steps 1 and 2.

1. Sign in to your GitHub account, and create a GitHub repo (such as named `homework`) on `GitHub` (https://github.com), you can add a README.md file or just choose not to add a README.md file.
2. On your local computer, open a `Git Bash` terminal.
3. Skip this step if you simply want the cloned repo to be in the current directory. Otherwise, In the terminal, type `mkdir myfolder` (create a folder named `myfolder` within the current directory) and then `cd myfolder` (change to the directory `myfolder`). The directory name `myfolder` can be any name you want.

4. `git clone https://github.com/Your_Git_UserName/homework.git` (change the remote repo path to match your actual remote repo).

   > **Note**
   >
   > To specify a specific folder to clone to, add the name of the folder after the repository `URL`, like this: `git clone github-repo-URL mylocalfolder`

5. Now you have established a connection between your local directory `homework` and the remote repo `homework` on GitHub.

6. Create a new file in the current local directory `homewor` on your local computer, such as using your favorite editor to create a file named `myfirstlocalfile.txt` with any content in it. Or for the sake of demonstration, you can use the following Linux command to create this file containing the line `#My first local file`.

   ```
   echo "#My first local file" >> myfirstlocalfile.txt
   ```

7. In the terminal, `git add .` This will add all changes to the **staging area**. This lets Git start to track the changes to files in your local directory.

8. Now you are ready to **commit** the changes, which versions (takes a snapshot of) the current files in the directory. A commit is a checkpoint where you can go back to.

   ```
   git commit -m "my first commit from local"
   ```

9. Now you are ready to sync the local repo with the remote repo.

   ```
   git push
   ```

   The GitHub might ask you to sign in for the first time. Choose `Sign in with your brower` to sign in to complete the push.

4.2.2 Initializing a Git Directory Locally First

The previous approach initializes a local Git repo by cloning a remote repo. You can also initialize a local Git repo by using `git init`. Follow the following steps:

3. Sign in to your GitHub account.

4. Create a GitHub **empty** repo (such as named `homework`) on `GitHub` (https://github.com) but make sure it is empty (do not add Readme.md file)

5. Start a Git Bash Terminal window on your local computer (You could also use the Terminal Window in RStudio or VSC). Navigate to the project directory; if you haven't yet created a project directory such as `homework`, do

   `mkdir project_dir` Example: `mkdir homework`

   Use `cd project_directory_name` to enter your local project directory;

   Use `ls` to list all files and directories or use `ls -al` to include all hidden files and directories. In your local Git Terminal, (note at this moment your local project directory is empty)

```
echo "# homework0" >> README.md  #create a file README.md
git init
git branch -M main #rename the branch name to main
git add .  # may use git add --all
git commit -m "first commit"
git remote add origin https://github.com/ywanglab/homework.git #(change the remote repo
git push -u origin main # only need to do this first time. Afterwards, only `git push`
```

   > **Note**
   >
   > 1. the general command format: `git push [remote-name] [branch-name]`
   > 2. difference between `git add .` and `git add --all`:
   >
   >   `git add .`: stages changes in the current directory and its subdirectories but does not include file deletions
   >
   >   `git add --all`: stages changes in the entire working tree, including deletions and untracked files. It is a more aggressive option and can be useful when you want to ensure that every change, including file deletions, is included in the next commit.
   >
   >   `git add --all` is equivalent to `git add -A`

6. if your local project directory already 1) contains files and 2) had performed `init git` before, then

```
git remote add origin https://github.com/ywanglab/homework.git` #(change the remote repo
git branch -M main
git push -u origin main
```

7. in the pop-out `GitHub Sign-in` window, click on `Sign in with your browser`.
8. Note an empty folder would not be pushed to the remote repo until it has a file (even an empty file) in it. In this case, you can create an empty file such as `.gitignore`

4.3 Some other common commands

1. check git status: `git status` and `git status --short` for a compact way.
2. `git commit -a -m "message"` will stage and commit every changed, already tracked file without using `git add changed_file`
3. `git add file_changed`

   # add `file_changed` to the staging environment, i.e., git repo to start track those changes.
4. use `git log` to check all commits. Use `git log --pretty=oneline` or just `git log --oneline` for shorter display.

   `git log origin/main` #check the remote repo `origin/main` commits
5. use `git diff origin/main` to show the differences between the local `main` and `origin/main`.
6. use `git checkout .` to revert back to the previous commit. Any changes after the previous commit will be abandoned.
7. to get to a previous commit, use `git checkout seven_character_commit_hash`. To get back to main, use `git checkout main`.
8. Git commit --amend

`commit --amend` is used to modify the most recent `commit`. It combines changes in the `staging environment` with the latest `commit`, and creates a new `commit`. This new `commit` replaces the latest `commit` entirely. Adding files with`--amend` works the same way as above. Just add them to the `staging environment` before committing.

One of the simplest things you can do with `--amend` is to change a `commit` message with spelling errors.

9. Git Revert HEAD:

   `revert` is the command we use when we want to take a previous commit and add it as a new commit, keeping the log intact. Revert the latest `commit` using git `revert HEAD` (`revert` the latest change, and then `commit`), adding the option `--no-edit` to skip the commit message editor (getting the default `revert` message):

   `git revert HEAD --no-edit`

> **Note**
>
> To revert to earlier commits, use `git revert HEAD~x` (*x* being a number. 1 going back one more, 2 going back two more, etc.)

10. Git Reset

    `reset` is the command used when we want to move the repository back to a previous `commit`, discarding any changes made after that `commit`. Let's try and do that with reset.

    ```
    git reset seven-char-commit-hash
    ```

11. Git Undo Reset

    Even though the commits are no longer showing up in the log, it is not removed from Git. If you know the commit hash you can reset to it:

    ```
    git reset seven-char-commit-hash
    ```

12. To permanently go back to a previous commit, use

    ```
    git reset --hard seven_char_commit_hash
    ```

13. to go back to a previous commit, but not changing the files in the working directory use the `--soft`‘ option.

    ```
    git reset --soft seven_char_commit_hash
    ```

14. `git remote -v` Get the reminder of the remote repo. To rename the remote origin: `git remote rename origin upsteam` rename remote repo `origin` to `upstream`

    > **Note**
    >
    > According to Git naming conventions, it is recommended to name your own repository `origin` which you have read and write access; and the one you forked for `upstream` (which you only have read-only access.)

15. if you want to remove the file only from the remote GitHub repository and not remove it from your local filesystem, use:

```
git rm -rf --cached file1.txt #This will only remote files; If intending to remove local file
git commit -m "remove file1.txt"
```

And then push changes to remote repo

```
git push origin main
```

14. For some operating system, such as Mac or Linux, you might be asked to tell GitHub who you are. When you are prompted, type the following two commands in your terminal window:

```
git config --global user.name "Your Name"
git config --global user.mail "your@email.com"
```

This will change the Git configuration in a way that anytime you use Git, it will know this information. Note that **you need to use the email account that you used to open your GitHub account**. `global` sets the username and e-mail for **every repo** on your computer. If you want to set the username/e-mail just for the current repo, remove `global`.

4.4 Use Git help

1. `git command -help` See all the available options for the specific command. Use '`--help` instead of `-help` to open the relevant Git manual page.
2. `git help --all` See all possible commands

4.5 When the upstream repo changes

When Git tells you the upstream repo is ahead,

15. Do `git pull` or `git pull origin`

    This is equivalent to `git fetch origin`, and then `git merge origin/main`.Then you can commit and push a new version to the remote repo.

16. `git pull` will not pull a new branches on the remote repo to local, but it will inform you if there is a new branch on the remote repo. In this case, just `git checkout the_remote_new_branch_name` will pull the remote branch to local. Note there is **no need** to create locally the branch by `git branch the_remote_new_branch_name`

4.6 Create branch

16. To add a branch to the main branch `git branch branchname`

    Switch the branch `git checkout branchname`

    To combine the above two actions, `git checkout -b branchname`, create a new branch named `branchname` if it does not exist and move to it.

Adding a file in branch `echo "#content" >> filename.txt`

Then `add` the file and commit the file. To push the branch to the remote repo we **have to use**

`git push --set-upstream origin branchname` The option `--set-upstream` can be replaced by `-u`

to see all branches in both local and remote: `git branch -a` Or `git branch -r` for remote only.

4.7 Merge branch to main branch

1. Switch from a branch (with name such as `branchname` to the `main` using
   `git checkout main`
2. on the `main` branch, Merge command to merge the branches
   `git merge branchname`

To delete a branch:

`git branch -d branchname`

4.8 Handle large files (>= 150Mb) on GitHub

GitHub does not allow to upload a file of size greater than 150Mb. However, one can use `git lfs` to handle large files exceeding this size up to several Giga bytes. The first thing is to install `git lfs`. Head to https://git-lfs.com, once dowlonad and install the Git command line extension, set up Git LFS for your user account by running

```
git lfs install  #(only need to do this the first time)
```

Then In each Git repository where you want to use Git LFS, select the file types you'd like Git LFS to manage (or directly edit your .gitattributes). You can configure additional file extensions at any time.

```
git lfs track "path/to/file"
```

Then do the regular `git add .` and `git -m "message"` and `git push`. Note one must use `git lfs track` a file first before doing `git add` and `git commit`.

> **Note**
>
> Note you need to track the large-size file first before you add it to the staging area. But often you will find this error after you try to push your changes to the GitHub. In this case, you will have to remove the commit history of this file first. One way to do this is to `reset -soft` the HEAD to the previous working HEAD, and then do `git lfs track` followed by `git add` and `git commit`, `git push`. Specifically,
>
> ```
> git reset --soft HEAD ~1  # or the_7-char_commit_hash
> git lfs track "path/to/large_file"
> git add .
> git commit -m "commit message"
> git push
> ```
>
> Note the `--soft` option allows the changes in the working directory not affected, otherwise any change after the previous commit will be removed.

4.9 Contribute by forking a GitHub repo and commit to the forked repo and create a pull request (refer to [the best workflow below]Section 6.3)

1. after forking a (foreign) GitHub repo to your own GitHub account, `git clone` that repo under your account to your local repo.
2. make changes in your local directory.
3. Submitting your changes for review

1. **Commit your changes locally.** Once you are ready to submit your changes, run these commands in your terminal:

   ```
   git add -A                              # Stages all changes, short for --all
   git commit -m '[your commit message]' # Makes a git commit
   ```

2. **Make a pull request.** (A pull request is a proposal to change) A GitHub pull request allows the owner of the forked upstream repo to review and make comments on your changes you proposed. Once approved, the upstream owner can merge your changes. Run:

   ```
   git push origin # Push current branch to the same branch on GitHub
   ```

4. Then go to your remote forked repo in your account on the GitHub site and click **Contribute**,and then **Open pull request**, this will take you to the upstream repo. In the form, leave a message explaining the change, and **Create pull request**. **Do not** select `Close pull request` unless you want to cancel the pull request.

4.10 Project

1. First make sure you have forked the course repo `https://github.com/ywanglab/stat1010.git` to your own GitHub account.
2. Now go to your GitHub account, git clone the forked course repo

`git clone https://github.com/your_git_user_name/stat1010.git`

to your local computer

4. add your resume file in the folder `./resume`

`git add`, `commit` and `push` your changes to the upstream repo using

```
git add .

git commit -m "added YourFirstName's resume"

git push origin
```

5. Then go to your remote forked repo in your account on the GitHub site and click **Contribute**,and then **Open pull request**, this will take you to the upstream repo. In the form, leave a message explaining the change, and **Create pull request**. **Do not** select `Close pull request` unless you want to cancel the pull request.

4.11 More on git

`git pull` = `git fetch` + `git merge`

- **`git fetch`** → downloads commits from the remote into your local refs (e.g. `origin/main`).
- **`git merge`** → merges those new commits into your current branch.

4.12 Git pull: What does `--ff` mean?

- `--ff` = **fast-forward if possible**.
- That means: if your branch has **no local commits** since it last matched the remote, Git will simply **move the branch pointer forward** to match the remote — no merge commit is created.

Example (before pull):

```
A---B---C  (origin/main)

A---B   (main)
```

If you run `git pull --ff` and your branch is strictly behind `origin/main`, Git just slides `main` forward:

```
A---B---C   (origin/main, main)
```

- `git pull` without flags:
  - May create a **merge commit** if histories diverged.
- `git pull --ff`:
  - Does a fast-forward if possible.
  - If not possible (you made local commits), Git falls back to a **merge commit**.
- `git pull --ff-only`:
  - Does a fast-forward **only**.
  - If not possible, it **aborts** with an error (no merge commit allowed).
- `--ff` is safe if you don't mind merge commits being created when necessary.
- `--ff-only` is stricter (no merge bubbles, linear history).

- Teams often configure one of these globally so `git pull` always behaves consistently.

when there is a diverge

- `--ff-only` → **aborts** with an error.
- `--ff` → falls back to a **merge**, creating a merge commit (see next section).

4.12.1 `git pull` or `git pull --ff` (merge fallback)

- Git fetches `origin/main` at `C`
- Git merges `C` into your local `main` with `D`, producing **M**:

```
After pull (local):
A B C
     \ \
      D M    (main)
        ^
     merge commit
```

When you run `git merge origin/main` (or `git pull` with merge strategy):

- Git identifies the **common ancestor** of the two branches → here, commit **B**.
- Then it looks at:
  - The changes between `B` → `C` (remote's changes).
  - The changes between `B` → `D` (your changes).
- Git tries to combine both sets of changes into a new snapshot.

That new snapshot becomes a new commit **M**. The merge commit **M** exists only **locally** until you `git push`. * When you **push**, `origin/main` is updated to point to **M**, and the remote history now includes that merge commit.

- Pros: Preserves exact history as it happened (no rewrite).
- Cons: Adds **merge commits**; history can get "braided".

4.12.2 Option 2: `git pull --rebase` (replay your work on top of remote)

- Git rewrites your local commits onto the fetched remote tip:
  - Rewrites `D` into `D'` applied after `C`.

```
A B C D'     (main)
      ^
     rebased (new) commit
```

- Pros: **Linear history**, no merge commit.
- Cons: **Rewrites** your local commits (new SHAs). If you had already pushed `D`, you'll need `git push --force-with-lease`, see below.

4.13 How to set opitons gloabally

- **Team prefers linear history** → `git pull --rebase` (and set it as default)

```
git config --global pull.rebase true
git config --global rebase.autoStash true
```

- **Keep exact history / avoid rewrite** → `git pull --ff` (merge when needed)

```
git config --global pull.rebase false
```

- **Never auto-merge; be explicit** → `git pull --ff-only`

```
git config --global pull.ff only
```

5 Concrete example: what does “merge C with D to produce M” look like?

Assume the repo has one file, `README.md`.

5.0.1 Commits and changes

- **B (common ancestor)** `README.md`:

  ```
  Hello project
  ```

- **C (remote, on origin/main)** — someone else added a line:

  ```
  Hello project
  Remote line
  ```

- **D (your local commit, on main)** — you added a different line:

  ```
  Hello project
  Local line
  ```

So history diverged:

```
A B C        (origin/main)
     \
      D        (main)
```

5.0.2 You run: `git pull` (merge strategy) or `git merge origin/main`

Git computes the diff **B→C** (“add `Remote line`”) and **B→D** (“add `Local line`”), applies both, and creates **merge commit M**:

- **M (merge result)**:

  ```
  Hello project
  Local line
  Remote line
  ```

(order may vary if both append—Git picks a consistent merge; if both edit *the same* line, you'll get a conflict to resolve.)

New history:

```
A B C
     \ \
      D M      (main)
```

M has **two parents**: D and C. That's a "merge commit".

6 3) What is `git push --force-with-lease` (and why it's safer than `--force`)?

When you **rebase** local commits that were already pushed, your local branch history no longer matches the remote's. A normal `git push` will be rejected. You need to overwrite the remote branch tip—i.e., a force push.

- **`git push --force`** overwrites the remote branch **unconditionally** (dangerous—you could clobber someone else's new commits if they pushed while you were rebasing).
- **`git push --force-with-lease`** is the **safe** version:
  - It says: "Force-push **only if** the remote branch still points to the commit I think it does."
  - If someone else has pushed new commits, **the push is rejected** instead of overwriting their work.

6.0.1 Typical rebase + push flow

```
# Update local view of remote
git fetch

# Rebase your local work onto the remote tip
git rebase origin/main   # resolve conflicts if any; git rebase --continue

# Safely update the remote branch
git push --force-with-lease
```

6.0.2 Example workflow with `git stash`

6.0.2.1 1. Check repo status

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
    modified:   app.py
    modified:   utils.py
```

You've made some edits but don't want to commit yet.

6.0.2.2 2. Stash your changes

```
$ git stash push -m "WIP: refactor utils"
Saved working directory and index state WIP on main: 1a2b3c4 Add logging
```

Now the working directory is clean.

6.0.2.3 3. Verify with `status`

```
$ git status
On branch main
nothing to commit, working tree clean
```

The changes are hidden away.

6.0.2.4 4. List stashes

```
$ git stash list
stash@{0}: On main: WIP: refactor utils
```

Your stash is safely stored.

6.0.2.5 5. Switch branch, pull, or do other work

```
$ git switch feature-branch
Switched to branch 'feature-branch'
```

6.0.2.6 6. Apply the stash back

```
$ git stash apply stash@{0}
On branch feature-branch
Changes not staged for commit:
    modified:   app.py
    modified:   utils.py
```

Changes are back, but the stash still exists in the list.

6.0.2.7 7. Or use `pop` to apply *and remove*

```
$ git stash pop
On branch feature-branch
Changes not staged for commit:
    modified:   app.py
    modified:   utils.py
Dropped refs/stash@{0} (abc123def456...)
```

6.0.2.8 8. Confirm stash list is empty

```
$ git stash list
# (no output - list is empty)
```

summary

- You edited files.
- `git stash push` cleaned your working directory but saved changes.
- Later, `git stash apply` or `git stash pop` restored those changes.

6.1 Rebuild the index respecting `.gitignore`

If you have modified `.gitignore` and you already pushed some files that you did not want to push, to remove those files already pushed to Github, you need to remove them from the git index to untrack them.

```
git rm -r --cached .  #redo all the index
git add .
git commit -m "Reindex: drop ignored files from repo"
git push origin <your-branch>
```

to remove specific folder or files:

```
git rm -r --cached .Rhistory .Rproj.user # `-r` is needed for a directory
```

6.2 Unstage and untrack

- **unstage** = remove from the staging area (index), but keep the file under Git's tracking.
- **untrack** = stop Git from tracking the file altogether.

6.2.1 To unstage (but keep tracking):

If you already ran `git add file.txt` and want to undo that:

```
git reset HEAD file.txt
```

Now `file.txt` is back in "modified" state but not staged. To unstage everything:

```
git reset HEAD
```

6.2.2 To unstage:

If a file is already committed to the repo but you want Git to forget it:

```
git rm --cached file.txt
```

- `--cached` removes it from the index (tracking) but leaves it in your working directory.
- Next commit will record the removal.

- If you want to untrack entire directories:

```
git rm -r --cached my_folder/
```

6.2.3 Prevent tracking in the future

Add the file or folder to `.gitignore` so Git won't pick it up again:

```
# .gitignore
file.txt
my_folder/
```

6.3 Best workflow with GitHub from Colab (or a local device)

Pre-req: Local repo is a clone of the GitHub repo with aligned HEAD

1. Keep sync with the upstream original owner repo. On GitHub, in the forked repo (under your account), Click on "Sync fork".
2. Open (or create) a notebook from G-drive to work with in Colab.
3. Then, mount the G-drive. If on a local device, use the same workflow (open or create a notebook from the project directory).
4. In a termnal of Colab (or a terminal in VSC in a local device)

- `git pull` or `git pull --ff` or (safer method: `git pull --ff-only`)

If permission denied on G-drive, run this first then repeat git pull.

```
chmod +x .git/hooks/*
```

3. After editing, and before finish

```
git status
git add  files-to-commit
git commit -m "commit message"
git push  # this will push the files-to-commit to your fork/main
```

6.4 Team Github workflow

6.5 Initial setup

1. Fork and Clone

- **Fork**: You click "Fork" on GitHub → it creates a **copy** of the repo under *your GitHub account*. Navigate to `https://github.com/ywanglab/STAT4160`, then click on "Fork".
- **Clone**: You download a local copy of *your fork* to your computer. (only do this for the first time)

So after forking, you typically do (only for the first time)

```
git clone https://github.com/YOUR-USERNAME/REPO-NAME.git #REPO-NAME should be STAT4160
cd REPO-NAME   # the REPO-NAME should be STAT4160, cd to the current working directory
```

2. Add the original repo as "upstream"

Your fork is linked to your GitHub account (the "origin"). To stay in sync with the original project, add a remote for the source repository:

```
git remote add upstream https://github.com/ywanglab/STAT4160
```

Check remotes:

```
git remote -v
# origin    https://github.com/YOUR-USERNAME/REPO-NAME.git (push/pull)
# upstream  https://github.com/ORIGINAL-OWNER/REPO-NAME.git (pull only)
```

3. Create a feature branch in your fork

Never work directly on `main`. Instead create a new branch:

```
git checkout -b feature/my-contribution  # eg: homework/your_initial
# edit files...

# after you done your edit, push changes to origin/main

git add files-to-commit  # git add filename (or directoryname) use `.` rarely as it will add
git commit -m "Fix bug in utils"
git push -u origin feature/my-contribution  #git push by default push changes to origin/main
```

4. Open a Pull Request (PR) (only do this for the contribution you want to make, such as homework)
5. Go to your fork on GitHub (`https://github.com/YOUR-USERNAME/REPO-NAME`).
6. GitHub will show a banner: "You recently pushed to `feature/my-contribution`. Do you want to open a Pull Request?"
7. Click it → select **base repository = (upstream) original owner repo**, **compare = your branch**.

Note: `head repository` → your fork (e.g. YOUR-USERNAME/REPO-NAME) and branch (feature/…) that contains your changes.

4. Write a good description and submit the PR.

Now the maintainers of the original repo will review it. If approved, they'll merge it.

6.6 Keep your fork in sync

Before making new contributions, update your `fork/main` with the latest `main` from upstream:

Option A) Do it on GitHub: If GitHub shows something like "This branch is 1 commit behind", "Sync Fork".

Option B): do it via terminal:

```
git checkout main   # checkout main
git pull upstream main        # pull from the upstream original repo
git push origin main          # update your fork on GitHub
```

Then branch off `main` again for your next feature.

6.7 Git FAQ

0) Explain staging area, working area, working directory

- **Working directory / working tree (aka "working area")**: The files on your disk you edit.
- **Staging area (index)**: The "snapshot-in-progress" you will commit next. You move changes here with `git add`.

- **Local repository (`.git`)**: The database of commits/objects/refs. `git commit` writes a new commit to this store.
- **HEAD**: A pointer to your current commit/branch.

6.7.1 Working directory (working tree) vs "actual files on disk"? Save vs commit? What are "index" and "working tree"?

- **Working directory/working files / working tree**: the files on your disk under the repo. This *is* the "actual files on disk" for the project (both tracked and untracked). What `git status` calls "Changes not staged for commit" (for tracked edits) and "Untracked files".
- **Index (staging area)**: a binary file at `.git/index` that holds the **exact snapshot** you will commit next. You put changes into the index with `git add`. `Git status` calls "Changes to be cmmitted".

Compare the layers

```
git status                          # see working tree vs index vs HEAD
git diff                            # working tree vs index: what you edited but haven't staged
git diff --staged                   # index vs HEAD: what's staged vs. last commit
git log --oneline --graph --decorate --all   # visualize history (merge vs rebase)
```

Flow:

```
(edit & save) → working tree
git add → index
git commit → new commit from the index
```

- **Local repository**: all Git objects in `.git/` (commits, trees, blobs, refs).

Save vs commit

- **Save**: editor/OS action that writes a file to disk (affects working tree only).
- **Commit**: Git action that records a snapshot of the **index** into the repository history (`.git/objects`) with a message and metadata.

6.7.2 1) After `git add`, how to undo (un-add) a file or directory?

Unstage (but keep your edits in the working tree):

```
# Preferred (Git 2.23+)
git restore --staged <file-or-dir>

# Older (still works)
git reset HEAD <file-or-dir>

# Unstage everything that's currently staged
git restore --staged .
# or
git reset  #eqiv to: git reset --mixed HEAD: reset the index to match the current HEAD (unsta
```

Partially unstage hunks:

```
git restore --staged -p <file>   # or: git reset -p <file>
```

If you accidentally started tracking something (e.g., should be ignored), remove it from the index only:

```
git rm --cached -r <path>   # leaves the file(s) on disk, stops tracking
```

6.7.3 2) After `git commit`, how to un-commit?

Undo the last commit locally (choose how much to keep):

```
git reset --soft HEAD~1   # keep changes staged
git reset --mixed HEAD~1  # keep changes in working tree (unstaged) [default]
git reset --hard HEAD~1   # discard the commit AND your local changes (danger!)
```

If the commit is **already pushed** (others may have pulled it), prefer:

```
git revert <commit-sha>   # makes a new commit that undoes the old one
```

Fix or edit the most recent commit without changing its parent:

```
git commit --amend
```

`git commit --amend` (more)

Rewrites the **last** commit.

- **Fix message only**:
  ```
  git commit --amend -m "Better message"
  ```
- **Add forgotten changes** (stage them first):
  ```
  git add <files>
  git commit --amend --no-edit   # keep prior message
  ```
- **Change author/committer timestamp**:
  ```
  git commit --amend --no-edit --reset-author
  ```

Results in a **new commit SHA**. If the old commit was pushed, you'll need:

```
git push --force-with-lease
```

If you *must* rewrite published history (e.g., after a local rebase), push safely:

```
git push --force-with-lease
```

6.7.4 3) When `git push`, what conflicts can occur? How to fix them?

A "push conflict" is usually a **non-fast-forward** rejection because the remote has new commits you don't have.

Symptom: `rejected] ... (fetch first)` or `non-fast-forward`.

Fix:

```
git fetch origin
# Option A: merge
git merge origin/<branch>

# Option B: rebase (keeps history linear)
git rebase origin/<branch>
```

```
# After Having Resolved any conflicts, then:
git push
```

6.7.5 4) When `git pull`, what conflicts can occur, and how to fix them?

`git pull` = `fetch + merge` (by default) or `fetch + rebase` (with `--rebase`). Conflicts occur when both sides changed the same lines or one side edits a file the other deleted.

Merge flow (default pull):

```
git pull
# If conflicts:
git status
# open files, resolve <<<<<<< ======= >>>>>>> markers
git add <resolved-file>...
git commit           # completes the merge
```

Rebase flow (`git pull --rebase`):

```
git pull --rebase
# If conflicts:
git status
# resolve, then:
git add <resolved-file>...
git rebase --continue
# or:
git rebase --abort    # to go back to the state right before rebase
```

Related:

- `git rebase --continue` → after resolving a conflict, proceed to the next commit.
- `git rebase --skip` → drop the problematic commit and continue.
- `git rebase --quit` → stop the rebase *without resetting your current files/HEAD*; it just removes rebase state (rarely needed—`--abort` is the safe "put it back" button).

Helpful:

```
git mergetool          # launch a diff/merge tool if configured
```

6.7.6 5) Why create a new branch instead of working on `main`?

- Keep `main` clean, stable, and deployable.
- Isolate work so you can open focused pull requests and get review.
- Parallel development without stepping on each other.
- Safer experiments; easy to abandon a branch if it doesn't pan out.
- Release/hotfix workflows (e.g., `release/*`, `hotfix/*`).
- CI/policy gates per branch.

6.7.7 6) How `git stash` works and why we need it

`git stash` saves your **uncommitted** changes (working tree and staged) into a stack entry, then reverts your tree to a clean state—handy when you must switch branches or pull/rebase but aren't ready to commit.

Common commands:

```
git stash push -m "wip: message"    # save staged + unstaged
git stash push -u                    # include untracked files
git stash push -a                    # include ignored files
git stash list
git stash show -p stash@{0}          # see what's inside
git stash apply stash@{0}            # apply, keep it on the stack
git stash pop stash@{0}              # apply and remove from the stack
git stash drop stash@{0}
# Partial / path-specific:
git stash -p                         # interactively stash hunks
git stash push -- <path1> <path2>    # stash only these paths
```

6.7.8 8) Difference between `git reset` and `git revert`

- **`git reset`**: Moves a branch/HEAD to another commit (optionally touching index and working tree). It **rewrites history** for that branch.
  - `--soft`: move HEAD only (keep index + working tree)
  - `--mixed` (default): move HEAD + reset index (keep working tree)
  - `--hard`: move HEAD + reset index + working tree (discard changes)
  - Use for local surgery (e.g., uncommit/squash) before sharing.
- **`git revert`**: Creates a **new commit** that undoes the changes from a prior commit. **Does not rewrite history**; safe on shared branches.

Rule of thumb: **Use `revert` for public history, `reset` for local/private history.**

6.7.9 9) How to remove files that are already pushed? Explain `git rm --cached`

If you only want Git to **stop tracking** the file(s) but keep them on disk:

```
git rm --cached -r <path>
echo "<path>/" >> .gitignore
git commit -m "Stop tracking <path>"
git push
```

> `git rm --cached` removes from the **index** (stops tracking) but **does not delete** your local copy.

If sensitive/big files are already in history and must be purged:

- Use **git filter-repo** (recommended) or BFG:

```
# after installing git-filter-repo
git filter-repo --path <path> --invert-paths
git push --force-with-lease --all
git push --force-with-lease --tags
```

- Rotate any exposed secrets and tell collaborators to re-clone or hard-reset to the new history.

6.7.10 10) Difference between `git pull --rebase` and `git pull -ff`

- **`git pull --rebase`**: Fetch, then reapply your local commits **on top of** the updated upstream. This rewrites your local commits for a cleaner, linear history. Configure permanently:

```
git config --global pull.rebase true     # always rebase on pull
# or for one repo:
git config pull.rebase true
```

  - `-f` is a short flag for **fetch** –**force**, so `-ff` is basically "fetch with force (twice)"".

- If you *don't* use `--rebase`, then `git pull` merges by default.`--ff-only` keeps history clean by aborting instead of making a merge commit when a fast-forward isn't possible.

6.7.11 11) Explain `git rebase`

Rebase "moves" your commits to a new base commit.

Example: keep a feature branch up to date without merge commits:

```
git checkout feature
git fetch origin
git rebase origin/main   # replay feature's commits on top of latest main
# resolve conflicts per-commit:
git add <resolved-file>...
git rebase --continue
# when done and if previously pushed:
git push --force-with-lease
```

Interactive rebase to clean history (reorder/squash/edit/drop):

```
git rebase -i HEAD~5
# pick | reword | squash | fixup | edit | drop
# tip: use autosquash:
git commit --fixup <sha>
git rebase -i --autosquash origin/main
```

Advanced: move a range of commits to a different base:

```
git rebase --onto <new-base> <old-base> <branch>
```

Guidelines

- Don't rebase commits others are already depending on (unless your team agrees and you use `--force-with-lease`).
- Test after rebases; conflicts are resolved commit-by-commit.

6.8 4) Difference between `git rebase` and `git merge`

Goal (both): bring changes from one line of history into another.

6.8.1 Merge

- Creates a **merge commit** that has two parents; preserves true history.
- Doesn't rewrite existing commits.
- Safer on shared branches; good for "what actually happened."

```
# Before
main:    A---B---C
             \
feature:       D---E

# Merge feature -> main
main:    A---B---C---M
                 /   \
feature:        D-----E
```

6.8.2 Rebase

- **Rewrites** your commits to appear on top of a new base (new SHAs).
- Produces a **linear** history (no merge commit).
- Avoid rebasing commits others already pulled (or force-push with care).

```
# Rebase feature onto latest main
main:    A---B---C
                  \
feature:           D'--E'    (D and E replayed; new SHAs)
```

Rule of thumb: merge for public/shared history; rebase to keep your feature branch tidy before sharing.

6.8.3 On which branch do `merge` and `rebase` happen?

- **`git merge other-branch`** merges *other-branch* **into the branch you currently have checked out** (the "current branch"). If you want to merge into some *target* branch, you must first switch to it:

  ```
  git switch target
  git merge other
  ```

- **`git rebase <upstream>`** rewrites the **current branch** so its commits replay on top of `<upstream>`:

  ```
  git switch feature
  git rebase origin/main
  ```

  Advanced: you *can* rebase a branch without checking it out:

  ```
  git rebase origin/main feature   # rewrites 'feature'
  ```

  But conceptually, rebase always **moves one branch's commits onto a new base**.

6.9 Quick reference (handy snippets)

```
# Unstage everything
git restore --staged .

# Uncommit but keep edits
git reset --mixed HEAD~1

# Undo a pushed commit safely
git revert <sha>
```

```
# Resolve pull with rebase and conflicts
git pull --rebase
# ...resolve...
git rebase --continue

# Stop tracking a file/folder (keep it locally)
git rm --cached -r <path> && echo "<path>/" >> .gitignore

# Fast-forward only pull (abort if divergence)
git pull --ff-only
```

6.10 1) Index vs. working files (aka working tree)

Working files / working tree

- The actual files on disk that you edit and save in your editor.
- Can include both **tracked** and **untracked** files.
- What `git status` calls "Changes not staged for commit" (for tracked edits) and "Untracked files".

Index / staging area

- A snapshot Git keeps (in `.git/index`) of **exactly what will be committed next**.
- You put changes into the index with `git add`.
- What `git status` calls "Changes to be committed".

Compare the layers

```
git diff             # working tree  vs index   (what you edited but haven't staged)
git diff --staged    # index         vs HEAD    (what's staged vs last commit)
```

Flow:

```
(edit & save) → working tree
git add → index
git commit → new commit from the index
```

6.11 3) "I saved a file on one branch, then checked out a new branch and edited it again. What version do I have on disk?"

It depends on whether your first edits were **committed** and whether switching branches would **overwrite** those edits.

6.11.1 Cases

1. **You did NOT commit, and the switch would overwrite your changes** Git **blocks** the switch:

   ```
   error: Your local changes to the following files would be overwritten by checkout:
     path/to/file
   ```

   Fix: commit, **stash**, or discard those changes first.

2. **You did NOT commit, and the switch does NOT overwrite your changes** Git **allows** the switch and carries your uncommitted edits into the new branch. On disk you see **your latest saved content** (not the branch's clean version). The changes now show as "modified" on the new branch. If you commit now, the commit lands on the **new branch**.

3. **You DID commit on the first branch** When you switch, Git rewrites your working tree to match the target branch's snapshot. You'll see the target branch's version of the file on disk.

4. **Untracked files** Untracked files follow you across branches. If an untracked path would conflict with a tracked file in the target branch, Git blocks the switch unless you stash with `-u` or clean with `git clean -fd` (dangerous).

6.11.2 Tips

- To keep branch changes separate, either commit/stash before switching or use **separate work trees**:

  ```
  git worktree add ../repo-main main
  git worktree add ../repo-feature feature
  ```

- To forcibly see a file as it exists on another branch (without switching):

  ```
  git show other-branch:path/to/file > path/to/file   # overwrites file on disk
  # or, with restore (safer semantics):
  git restore --source other-branch -- path/to/file
  ```

6.12 4) Suggested team workflow (you maintain `main`, teammates contribute)

Below is a light-weight, reliable **feature-branch + PR** flow (GitHub/GitLab/Bitbucket compatible).

6.12.1 Repository / policy (one-time setup)

- **Protect `main`**: disallow direct pushes, require PRs, require at least 1 review, require CI to pass, and (optionally) **Require linear history**.
- Prefer **"Squash and merge"** or **"Rebase and merge"** on PRs to keep `main` tidy.
- Add CODEOWNERS (optional) so certain paths require your review.
- Encourage small, focused PRs.

6.12.2 Personal Git config (everyone)

```
git config --global pull.rebase true     # rebase on pull; cleaner history
git config --global fetch.prune true     # remove deleted remote branches on fetch
git config --global rerere.enabled true  # remember conflict resolutions (handy)
```

6.12.3 Contributor workflow (feature branch)

```
# 1) Sync and branch off up-to-date main
git switch main
git fetch origin
git pull --ff-only       # keep local main as a clean fast-forward
git switch -c feature/short-desc

# 2) Develop
# ...edit, test, commit in small logical chunks...
git add -p
git commit -m "feat: short message"

# 3) Keep branch current (periodically)
```

```
git fetch origin
git rebase origin/main   # replay your commits on latest main
# resolve conflicts → git add ... → git rebase --continue

# 4) Publish and open PR
git push -u origin feature/short-desc
# (Open PR, link issue, ensure CI passes, request review)

# 5) Address review
# Use fixup commits for clean history:
git commit --fixup <sha-to-fix>
git rebase -i --autosquash origin/main
git push --force-with-lease
```

6.12.4 Maintainer (you) merging PRs

- Ensure tests pass, reviews done.
- Choose **Squash & Merge** (one clean commit on `main`) or **Rebase & Merge** (preserve individual commits but linear).
- After merge:

  ```
  # Keep your local main clean and current
  git switch main
  git pull --ff-only
  ```

- Optionally tag releases:

  ```
  git tag -a v1.2.3 -m "Release 1.2.3"
  git push origin v1.2.3
  ```

6.12.5 Hotfixes

- Branch from `main`: `git switch -c hotfix/issue-123`
- Patch, test, PR, merge → tag a patch release.

6.12.6 Common "gotchas" and fixes

- **Push rejected (non-fast-forward)**: `git fetch origin && git rebase origin/main` (then resolve & push).
- **Rebased your feature and need to update PR**: `git push --force-with-lease`.

- **Can't switch branches due to local edits**: commit, `git stash` (use `-u` to include untracked), or discard.

6.12.7 Quick reference of commands mentioned

```
# See differences between layers
git status
git diff
git diff --staged

# Stage/unstage in parts (hunks)
git add -p
git restore --staged -p <file>

# Stash changes
git stash push -m "wip"        # tracked files
git stash push -u -m "wip"     # include untracked
git stash list
git stash show -p stash@{0}
git stash pop                  # apply & drop top entry

# Safe push after history rewrite
git push --force-with-lease
```

6.13 How to fix conflict when switching branches

Conflicts occurs when two branches have different versions (commit HEADs) of a file, and you then edit one version of that file on one branch without commiting it, and then you try to switch to another branch. Git will block you switching. When both branches point to the same version, then no conflicts arises and the change in one branch follow you to a new branch.

Conflicts typically occrurs when 1) same hunk edited differnetly in both branches (overlapping lines). Git typically will merge differnces of two different lines. 2) delete/modfify: one side deleted a file, the other edited it. 3) rename/rename to diffent names.

Fix options: * Keep your WIP for later

```
git stash push -m "WIP" # Git saves the changes on the branch where you made changes (but not
git checkout main #switch to a different branch
git stash pop  # apply the changes and delete the stash on main. You can do this on any branc
```

- or commit your WIP on the current branch (where you made the change), then switch

```
git add -A
git commit -m "WIP"
git checkout main #switch
```

- or discard WIP (dangerous), put files back to the last commit

```
git restore notes.txt # perform on the branch where you made the change.
git checkout main
```

Rule of thumb

Stash: Do it on the branch where your changes currently live, but you can apply later anywhere.

Restore: Do it on the branch where you want to discard/reset changes (usually the branch you're already on).

6.13.0.1 When Git cannot auto-merge, how to resolve a conflict

Open the file → delete conflict markers → keep desired content->Save

git add to mark resolved.

git commit or git stash drop (mark it resolved witout commit, and drop the stash if no longer need it)

6.14 When conflict occurs using git pull and how to resolve it

This is when remote repo and local repo diverges. (remote repo and local repo share a common ancetor, but each has new commits.) If a conflict occurs, git pull will not make a merge commit but will merge all files without conflicts.

If using **Merge flow**

```
git pull # (fetch+merge: fetch updating the index of origin/main, always successful. merge al
# if conflicts:
# 1) edit files to remove <<<<<<< ======= >>>>>> markers
git add <files>
git commit        # completes the merge commit
# or bail out:
git merge --abort # moving back to where it was before git merge (the updated remote index ke
```

If using **Rebase flow**:

```
git pull --rebase
# if conflicts:
# 1) fix files
git add <files>
git rebase --continue

git push --force-with-lease # only needed if your rebase rewrote history(commits) already on
# or bail out:
git rebase --abort
```

If **Parking your work**

```
git stash -u      # include untracked. -a (aka --all) including ignored.
# ... switch branches / pull ...
git stash pop     # reapply; resolve if conflicts
```

6.15 Conflicts occur when git push and how to resolve

The conflicts may occur due to several situations:

1) Non-fast-forward push (someone pushed before you) **Fix A: Merge approach (simple)**

```
git pull             # resolve conflicts if prompted
git push
```

Fix B: rebase apprach (cleaner history)

```
git fetch
git rebase origin/main
# if conflicts: edit files → git add <files> → git rebase --continue (repeat)
git push
```

2) Push rejected after you rewrote history (by amend/rebase) What is rewriting history? an operation changes the existing commit ID (SHA)

- git commit –amend
- git rebase
- git reset –hard followed by further commits.
- History-editing toosl (git filter-repo, etc)

A typical way to use git commit –amend are: 1. add a missing file

```
git add missing.file
git commit --amend --no-edit
```

2. edit commit message

```
git commit --amend -m "new message"
```

```
# You amend or rebase (history changes)
git commit --amend --no-edit  #--no-edit: keep the same commit history
git push
# ! [rejected] (non-fast-forward)
```

Then

```
git push --force-with-lease
```

3) No upsteam branch

```
git push -u origin feature/api #simply use -u to create the new branch
```

4) rejectd because the branch is a protected branch. Create PR.

5. largile/file type blocked (server or hooks) Fix: Use Git LFS:

```
git lfs install
git lfs track "*.mp4"
git add .gitattributes bigvideo.mp4
git git commit -m "Track with LFS"
git push
```

6.16 Git merge a branch

- Whole branch → git merge feature-x
- One commit → git cherry-pick
- One file/dir → git restore –source feature-x –
- Single-commit result → git merge –squash feature-x # only contents from feature-x, no merge commit history.

In-merge resolution → –ours/–theirs (per-file) or -X ours/theirs (strategy)

```
# keep your current branch's version for that file
git checkout --ours  path/to/file

# keep the merging branch's version for that file
git checkout --theirs path/to/file

git add path/to/file


# prefer current branch on conflicts
git merge -X ours feature-x

# prefer the other branch on conflicts
git merge -X theirs feature-x
```

6.17 resolve a file conflict after `git pull`:

6.18 1) See what's conflicted

```
git status
git diff --name-only --diff-filter=U   # lists just unmerged files
```

6.19 2) Open the conflicted file and resolve

You'll see markers like:

```
<<<<<<< HEAD
# your local version
=======
# incoming version (from remote)
>>>>>>> origin/main
```

Edit the file so it has the content you want, **delete the conflict markers** (`<<<<<<<`, `=======`, `>>>>>>>`), and save.

- With vim (side-by-side):

  ```
  vim -d path/to/file
  ```

  or open normally:

  ```
  vim path/to/file
  ```

6.20 3) Mark it resolved and continue

```
git add path/to/file
```

Now finish the operation depending on what `pull` was doing:

- **If it was a merge** (default `git pull`):

  ```
  git commit   # completes the merge (pre-filled message)
  ```

- **If it was a rebase** (`git pull --rebase` or configured):

  ```
  git rebase --continue
  ```

- (If you decide to bail out)

  ```
  git merge --abort      # during a merge
  git rebase --abort     # during a rebase
  ```

6.20.1 Fast shortcuts (use with care)

- **Keep your version entirely**:

```
git checkout --ours path/to/file    # during merge
git add path/to/file
```

- **Take the remote version entirely**:

```
git checkout --theirs path/to/file  # during merge
git add path/to/file
```

Note: In a **rebase**, "ours/theirs" feel inverted (because your commits are being replayed). If unsure, inspect with `git show :1:path/to/file :2:path/to/file :3:path/to/file` or just open the file and edit manually.

6.20.2 Sanity checks

```
git status                  # no more "both modified"
git diff --check            # finds leftover conflict markers
```

That's it: resolve → `git add` → finish (`git commit` or `git rebase --continue`).

6.20.3 B) Abort the in-progress operation, then pull clean

If you don't want to resolve right now:

```
git merge --abort    # if a merge was in progress (ignore if says none)
git rebase --abort   # if a rebase was in progress
```

Now choose:

- **Keep your local edits but update from remote**:

```
git stash -u
git pull --rebase
git stash pop      # resolve any conflicts, then: git add ...; git rebase --continue
```

- **Discard local edits and match remote exactly (DANGEROUS)**

```
git fetch origin
git reset --hard origin/$(git branch --show-current)
# now pull will be clean
```

6.21 Steps to fix conflict:

1. Make a quick safety back branch (optional) `git branch backup`
2. Fetch latest from origin `git fetch origin`
3. See the divergence (Optional) 'git rev-list –left-right –count (**u?**)…HEAD # output like: "3 2" means: 3 commits behind, 2 commits ahead (diverged)

- **`git rev-list`** Lists commit objects in reverse chronological order.
- **`--left-right`** When comparing two commit ranges, marks commits as:
  - `<` (left side) if reachable from the first commit set
  - `>` (right side) if reachable from the second commit set
- **`--count`** Instead of listing commit hashes, just show how many commits are on each side.
- **`@{u}`** Shorthand for "upstream" of the current branch (usually `origin/main` or `origin/<branch>`).
- **`...` (triple-dot)** Symmetric difference: `A...B` means "commits reachable from A or B but not both." So `@{u}...HEAD` means "commits that are either only in upstream or only in HEAD (local branch), but not in both."

So the full command counts how many unique commits are on:

- the **left** (`@{u}` = upstream),
- the **right** (`HEAD` = your local branch), in their symmetric difference.

Example output

```
$ git rev-list --left-right --count @{u}...HEAD
3       2
```

Interpretation:

- `3`: commits upstream has that your local branch does not (you're **behind** by 3).
- `2`: commits your local branch has that upstream does not (you're **ahead** by 2).

So your branch has diverged: upstream got 3 new commits, and you added 2 different ones.

6.22 Option A: Rebase (keeps linear history)

This replays your local commits on top of the remote branch.

```
# Rebase your current branch onto its upstream
git rebase @{u}
# If conflicts appear: fix files, then:
git add <fixed-files>
git rebase --continue
# If it gets messy:
git rebase --abort   # to go back to before the rebase
```

When done, push (may require `--force-with-lease` because history changed):

```
git push --force-with-lease
```

Make this your default pull behavior (repo-local):

```
git config pull.rebase true
# or globally:
git config --global pull.rebase true
```

6.23 Option B: Merge (creates a merge commit)

This preserves both histories and adds a merge commit.

```
git merge @{u}  #if no conflic files, then this should fix the problem already.
# If conflicts appear: fix files, then:
git add <fixed-files>
git commit   # completes the merge
git push
```

Make this your default pull behavior (repo-local):

```
git config pull.rebase false
# or globally:
git config --global pull.rebase false
```

6.24 Option C: Fast-forward only (fail if divergence exists)

Useful when you **never** want merges/rebases via `pull`:

```
git pull --ff-only
# or set as default
git config pull.ff only
# (global)
git config --global pull.ff only
```

6.25 "I just want to discard my local commits and match remote"

Warning: This throws away your local commits on this branch.

```
git reset --hard @{u}
```

6.26 Workfow when devloping codes on a local computer:

The codebase is in `main`, and revising code in branch `dev`. Then I make a commit on `dev`. Now we update `main` with the new commit of `dev`.

6.26.1 most common case: fast-forward

If `main` hasn't moved since you branched:

```
git status                          # make sure you're clean; stash/commit if not
git switch main
git log --oneline main..dev         # review what will come in
git merge --ff-only dev             # fast-forward main to dev's commit(s)
### optional: delete the feature branch
git branch -d dev
```

The workflow works when there is no diverge: `main` is ancestor of `dev`

6.26.2 If `main` and `dev` diverged

You have two good choices:

A) Merge (keeps both histories)

```
git switch main
git merge dev                       # resolve conflicts if any
# (edit files) -> git add -A -> git commit
```

B) Rebase then fast-forward (linear history)

```
git switch dev
git rebase main                     # replay dev on top of main; resolve conflicts
git switch main
git merge --ff-only dev             # now a fast-forward
```

6.26.3 Notes

- If you truly want `main` to **exactly match** `dev` (and don't care about preserving `main`'s extra commits), you can do:

  ```
  git switch main
  git reset --hard dev              # moves main pointer to dev (history rewrite)
  ```

- Always make sure your working tree is clean first (`git status`). If not:

  ```
  git stash -u -m "save work"       # later: git stash pop
  ```

6.27 `git add`

```
git add -A      # everything (adds/mods/deletes, whole repo)
git add -u      # mods + deletes (tracked only, no new files. `-u`: update)
git add .       # adds/mods here; ignore deletions outside and inside the current directory.
git add -p      # interactive (hunks)
```

Other handy flags:

- `--squash`: bring in all changes as **one** staged change (no merge commit, not a real merge).

```
git merge --squash dev && git commit -m "Squash dev"
```

- `--no-commit`: stage the merge result but stop before creating the merge commit (lets you review/tweak).

6.27.1 rebase

`rebase` rewrites the **current** branch's commits to a new base. You rebase the branch that you want to "move":

```
Before:
main: A   B   C
dev:       \  D   E

git switch dev
git rebase main   # replay D,E on top of C

After:
main: A   B   C
dev:              D'   E'
```

Now `main` can fast-forward to `dev`:

```
git switch main
git merge --ff-only dev
```

Rebasing `main` onto `dev` would rewrite `main` (risky, especially if shared). Safer pattern: **rebase the feature branch, not the trunk**.

7 5) `git stash -u -m "message"` and `git stash pop`

- `git stash` saves your **working tree** (and by default the **index**) to a stack, and reverts your tree to a clean state.`stash` is repo-wide, nto tied to a branch.
- Flags:
  - `-u` / `--include-untracked`: also stash **untracked** files (but not ignored).
  - `-m "message"`: label the stash (nice when you have many).
  - Modern syntax uses `stash push` (equivalent to plain `stash`):

```
git stash push -u -m "WIP parser refactor"
```

- **Apply vs Pop**
  - `git stash apply [<stash>]`: apply changes **but keep** the stash in the stack.
  - `git stash pop [<stash>]`: apply changes **and remove** that stash entry if the apply succeeds.

```
git stash pop          # takes the latest stash, applies, then drops it
```

  - If there are **conflicts**, you'll resolve them, then continue as usual. If a pop fails partway, the stash may **not** be dropped—check `git stash list`. `git stash apply` applies the selected stash onto whatever branch you are on when you run it.
- Useful extras:

```
git stash list                    # see stashes
git stash show -p stash@{2}       # see what's inside
git stash drop stash@{2}          # delete a specific stash
git stash clear                   # remove all stashes (careful!)
git stash push --keep-index       # stash only unstaged; keep staged work
git stash push -p                 # interactively choose hunks to stash
git stash branch fix-wip stash@{0}  # make a new branch from a stash
```

In Git, the **index** (aka the **staging area**) is the middle layer between your working directory and your last commit (`HEAD`). Think of it as your **shopping cart** for the next commit: whatever is in the index is exactly what `git commit` will record.

7.1 Mental model (the "three trees")

- **Working tree**: files on disk you edit.
- **Index / Staging area**: a snapshot you're preparing.
- **HEAD**: the last committed snapshot.

7.2 How it's used

- `git add <path>` → copies the file's current content from working tree **into the index**.
- `git commit` → writes a new commit **from the index** (not directly from your working files).
- `git restore --staged <path>` or `git reset <path>` → remove/rewind paths **from the index** (without touching the working file).
- `git rm` → removes a file from working tree **and** stages its deletion in the index.

7.3 Diff cheat-sheet

- `git diff` → working tree **vs** index (what you changed but haven't staged).
- `git diff --staged` (or `--cached`) → index **vs** HEAD (what will be committed).

7.4 Merges & conflicts

During a merge/rebase, conflicts show up as **unmerged entries** in the index (with stages):

- stage 1 = base, 2 = "ours" (HEAD), 3 = "theirs".
- See them: `git ls-files -u`.
- Resolve by editing files, then `git add` to update the index with your resolution.

7.5 Partial commits

- `git add -p` lets you stage **only some hunks**, building a clean, focused commit from the index.

7.6 Where it lives / advanced bits

- It's a binary file at `.git/index` caching file paths, blob IDs, and file stat info (for fast change detection).
- Power tools: `git update-index`, flags like `--assume-unchanged` / `--skip-worktree` (advanced, use sparingly).

Bottom line: The index is "what your next commit will be." Use `add`/`restore`/`reset` to curate it; use `diff`/`diff --staged` to inspect it.

7.6.1 Explain `git restore --staged <path>` or `git reset <path>`

Both commands are about the **index (staging area)** for the given `<path>`:

- **`git restore --staged <path>`** Unstages `<path>` by resetting the **index** copy of that file to match **HEAD** (the last commit). **HEAD: unchanged**, **index: reset to HEAD**, **working tree: untouched**.
- **`git reset <path>`** (shorthand for `git reset --mixed HEAD -- <path>`) Does the **same thing** as above: reset the **index** entry for `<path>` to match **HEAD**; leave your working file alone. Same effects as `restore --staged`.

Think of both as: "I staged something by mistake; put the staged version back to what's in the last commit, but don't touch my edits on disk."

7.7 Common scenarios & exact commands

7.7.1 1) I ran `git add -A` by accident and staged too much

```
git restore --staged path/to/file      # new-style, explicit
# or:
git reset path/to/file                 # classic equivalent
```

Your edits stay in the working directory; they're just **no longer staged**.

7.7.2 2) I staged a deletion, but I want the file back

Unstage the deletion **and** bring the file back to disk:

```
git restore --staged path/to/file
git restore path/to/file                # restores the working tree copy from HEAD
```

7.7.3 3) I want the index for this file to match some other commit (not HEAD)

```
git restore --source=<commit> --staged path/to/file
# or:
git reset <commit> -- path/to/file
```

Useful to "revert just this file" to an older version (you'd then `git commit` that change).

7.7.4 4) I want to discard *both* staged and working-tree changes (revert file fully)

```
git restore --source=HEAD --staged --worktree path/to/file
# classic (dangerous if you typo the path):
git checkout -- path/to/file   # older porcelain that touches worktree too
```

7.7.5 5) Interactive unstage by hunks

```
git reset -p path/to/file             # unstage selected hunks
# (newer porcelain also supports)
git restore -p --staged path/to/file
```

7.8 When to prefer which

- Use **`git restore --staged`** for clarity (it cannot move HEAD/branches; it only touches the index).
- **`git reset <path>`** is ubiquitous and shorter, but **`reset`** also has *other* modes (`--soft`/`--hard`, moving HEAD), so it's easier to misuse.

7.9 Quick reference table

Command	HEAD	Index (staging)	Working tree
`git restore --staged <path>`	same	**to HEAD**	same
`git reset <path>`	same	**to HEAD**	same
`git restore <path>`	same	same	**to HEAD**
`git restore --staged --worktree <path>`	same	**to HEAD**	**to HEAD**
`git reset <commit> -- <path>`	same	**to <commit>**	same
`git reset --hard` (no path)	move	**to new HEAD**	**to new HEAD**

Bottom line: Use either `git restore --staged <path>` or `git reset <path>` to **unstage** without losing your edits. Add `--source=<commit>` if you want the index to match some other commit.

8 Hwo to go back to a previous commit?

9 1) Just look around at an old commit (temporary)

```
git log --oneline                # find the commit hash (e.g., a1b2c3d)
git switch --detach a1b2c3d      # or: git checkout a1b2c3d
# You are in DETACHED HEAD (not on a branch)
# Go back to where you were:
git switch -                     # or: git switch <your-branch>
```

10 2) Make a new branch starting from that commit

```
git switch -c debug-old a1b2c3d  # or: git checkout -b debug-old a1b2c3d
```

11 3) Restore a single file as it was in that commit

```
git restore --source=a1b2c3d path/to/file
# (older git)
# git checkout a1b2c3d -- path/to/file
```

12 4) Move the current branch pointer back (rewrite history)

Dangerous if already pushed.

```
git reset --hard a1b2c3d
# If remote already has newer commits, you'd need:
git push --force-with-lease
```

13 5) “Undo” by adding inverse commits (safe for shared repos)

```
git revert a1b2c3d                # revert one commit
git revert <old>..HEAD            # revert a range (excludes <old>, reverts after it)
```

14 My Jupyter Notebook

Yi Wang (boldfaced using ** **)

Educator AUM

The following line is italicized using * *

I am interest in data science because it is a discipline that I feel love with.

14.0.1 Perform addtion

```
# code block
1+1
```

2

14.0.2 Horizontal Rule

Three or more

first rule using ***

---

using dashes —

Using (underscores) ____

14.0.3 Bulet list

using *

- Bird
- Frog
- Cat
- Dog

14.0.4 Numbered list

using 1. item (there is a space between 1. and item)

1. Apple
2. Pear
3. Peach

14.0.5 Tables

left-aligned	centered	right-aligned
1/2/2020	Mary	Apple
1/3	Johnason	Tomato

14.0.6 Hyperlinks

Click here to access my github account.

14.0.7 Images

Figure 14.1: A computer monitor

14.0.8 Code/Syntax highlighting

```
s = "Python syntax highlighting"
print s
```

14.0.9 Blocked quotes

using >

> Blockquotes are very handy in email to emulate reply text.
>
> This line is part of the same quote.

14.0.10 Strikethrough

using ~~ before and after a phrase

~~strikethrough this~~

15 Homework Assignments

I will use some assignments from https://cognitiveclass.ai.

1. Browser `Course & Projects`. Search for `Python for Data Science`. `Enroll Now` the class, and `Go to the Course`, and `Start the Course`.

2. Complete the following assignments from `Modules 1-4` and Part of `Module 5`. Excluding the `API` section in `Module 5`.

Module	Contents	Suggested Deadlines
Module 1	Python Basics	10/09/2023
Module 2	Python Data Structures	10/09/2023
Module 3	Python Prrogramming Fundamentals	10/09/2023
Module 4	Working with Data in Python	10/16/2023
Module 5	Working with Numpy Arrays (Excluding Simple APIs)	10/16/2023
Final Exam	Optional	

   Complete all `Practice Questions`, `Review Questions` and `Labs`. After your completing all the assignments, click on `Progress`, print the page (in PDF or hard copy), and send it to me. The page should show your `username` on the top right corner.

3. Enroll in the course `Data Anlaysis with Python`.

   Complete the following assignments.

Module	Contents	Suggested Deadlines
Module 1	Introduction	10/23/2023
Module 2	Data Wrangling	10/30/2023
Module 3	Exploratory Data Analysis	11/06/2023

4. Enroll in the course `Data Visualization with Python`.

   Complete the following assignments.

Module	Contents	Suggested Deadlines
Module 1	Introduction to Visualization	11/13/2023
Module 2	Basic Visualization Tools	11/20/2023
Module 3	Specialized Visualization Tools	11/27/2023
Module 4	Advanced Visualizaiton Tools (Optional)	

References